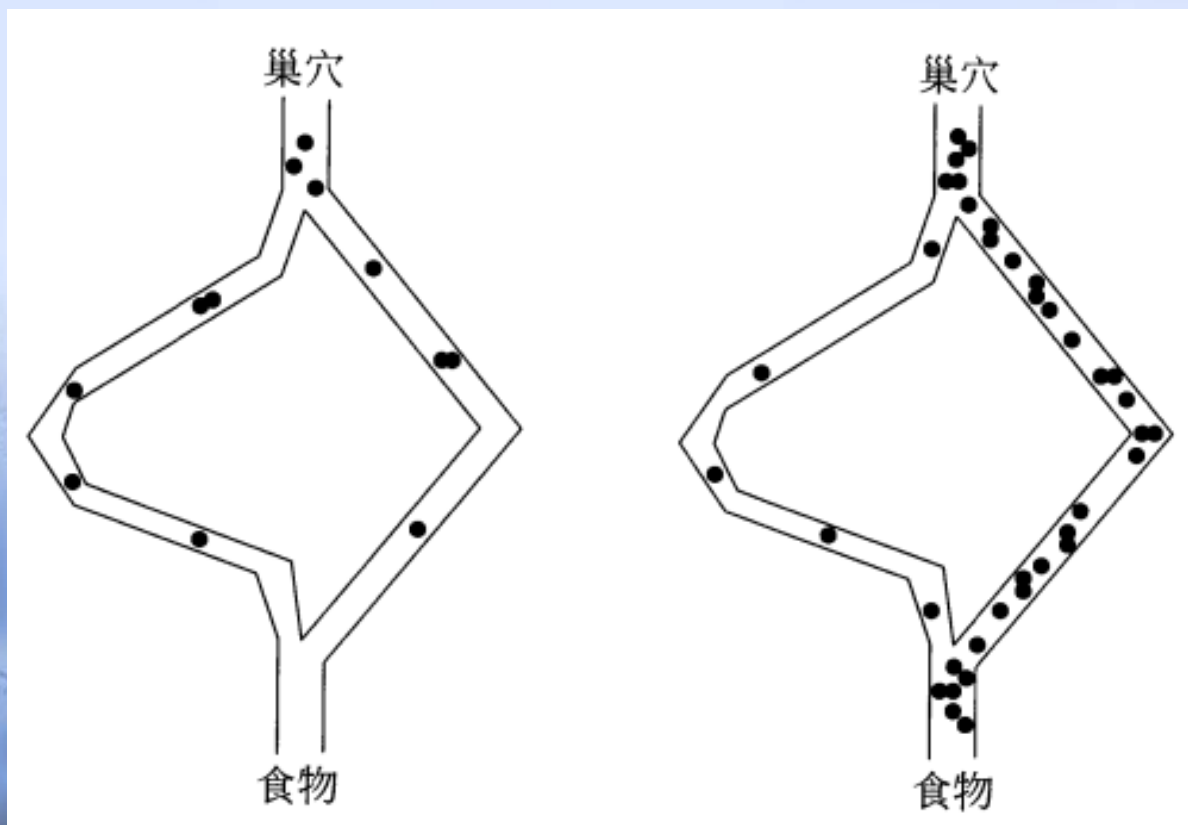




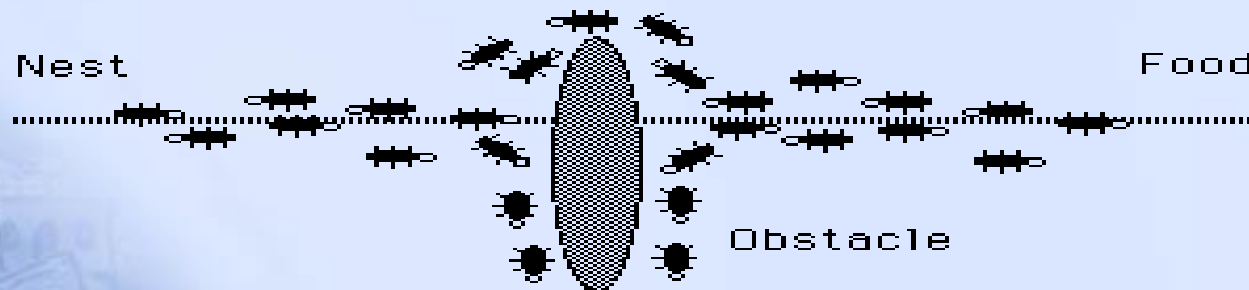
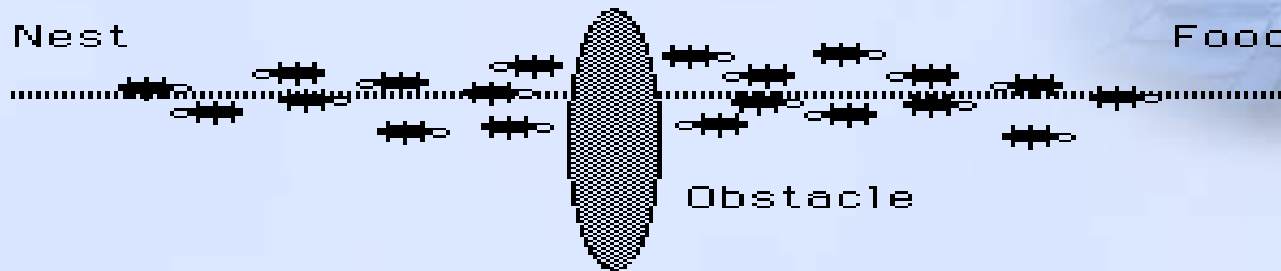
蚁群优化算法

蚁群优化算法的思想来源于蚂蚁觅食的群体行为
(ant colonies' foraging behavior)



群体智能

- 自然界蚂蚁、蜜蜂、细菌等都是群体生物, **群体中表现出来的复杂的行为**是一种生存的策略。这种复杂行为是通过简单个体的交互过程突现出来的**群体智能**。
- 群体中每个个体都能够改变环境, **个体的能力或行为规则非常简单**, 因而群体智能的实现比较方便, 具有**简单性的特点**。
- 群体中的这些个体之间是一种**间接通信的方式**。由于群体智能可以通过非直接通信的方式进行信息的传输与合作, 因此**具有较好的可扩充性**。某一个或几个个体出现故障不会影响群体对整个问题的求解。



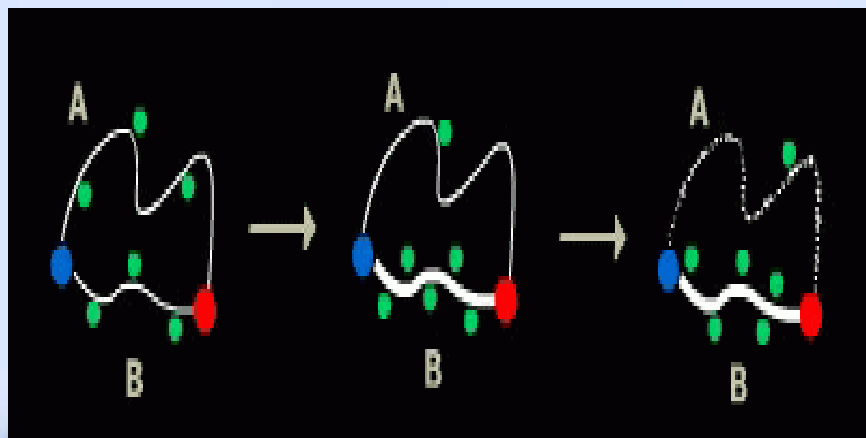
蚁群优化算法

- 几乎没有视觉感知系统，单个蚂蚁不能完成复杂的任务。
- 通过群体间的交流能够实现有纪律、有秩序地搬运食物。
- 群体的生存智能依赖于一种化学元素—信息素 (pheromone)
- 利用群体产生的信息素痕迹，建立从食物到巢穴之间的最短路径

信息素踪迹

- 每个蚂蚁都是随机选择路径
 - 蚂蚁行进时，会在路径上释放信息素，作为群体内间接通讯的物质。
 - 蚂蚁通过感知地面上的信息素浓度，倾向于往信息素高的路径上行进。
 - 一段时间以后，由于较短路径上往返的蚂蚁较多，信息素积累量较多，就会有更多的蚂蚁选择该路径，形成最短路径以后，所有蚂蚁都会在该路径上行走。
- 是一种自组织的正反馈机制。

自然界：蚂蚁会怎么做？



□ 路径 **B** 比较短, 蚂蚁完成这段路上的时间较少, 相应的释放在该路径上的信息素就较多。

- 路径**B**上的信息素浓度增加较快, 路径**A**上增加较慢, 当路径**B**上的信息素明显高于路径**A**时, 原来在路径**A**上的蚂蚁会放弃路径**A**, 重新选择路径**B**。
- 这样, 路径**B**上的蚂蚁越来越多, 当大部分蚂蚁都在路径**B**上行进时, 路径**A**的信息素会逐渐蒸发, 直至完全消失, 最后只留下路径**B**。

蚁群优化算法原理

表 5.1 蚁群觅食现象和蚁群优化算法的基本定义对照表

蚁群觅食现象	蚁群优化算法
蚁群	搜索空间的一组有效解(表现为种群规模 m)
觅食空间	问题的搜索空间(表现为问题的规模、解的维数 n)
信息素	信息素浓度变量
蚁巢到食物的一条路径	一个有效解
找到的最短路径	问题的最优解

蚂蚁算法

- AS (Ant System , AS)是蚁群算法的雏形，为解决旅行商问题 (Traveling Salesman Problem, TSP)而提出的。

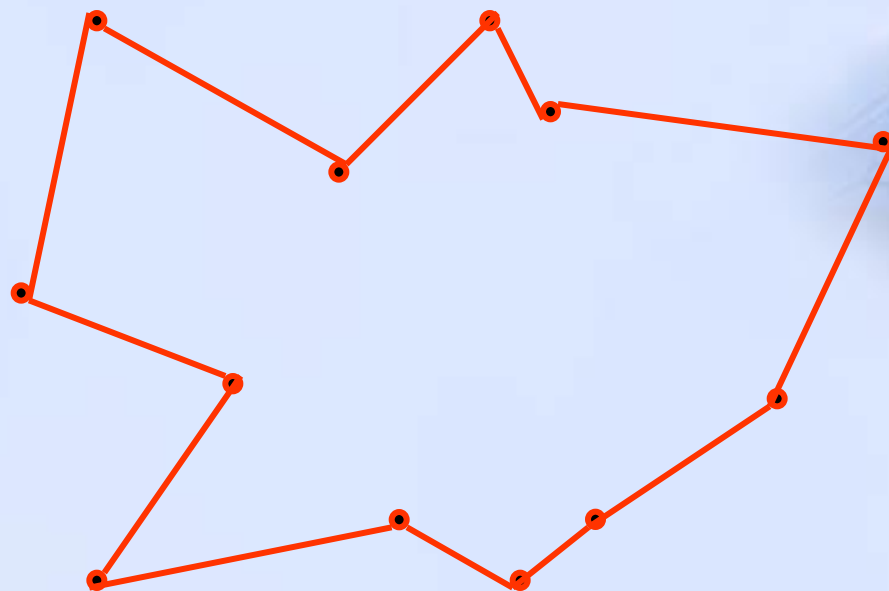
AS的出现为各种典型的改进算法的诞生提供了灵感：

- 精华蚂蚁系统 (Elitist AS, EAS)
- 基于排列的蚂蚁系统(rank-based AS, AS_{rank})
- 最大最小蚂蚁系统 (MAX-MIN AS, MMAS)

它们大多是在 AS 上直接进行改进，通过修正信息素的更新方式和增添信息素维护过程中的额外细节。

- 蚁群系统(Ant Colony System , ACS),
- 并行蚁群算法(Parallel Ant Colony System , PACS)
- 连续蚁群算法(Continuous Ant Colony System , CACS)

蚂蚁系统求解TSP



已知 n 个城市的集合 $C_n = \{c_1, c_2, \dots, c_n\}$, 任意两个城市之间均有路径连接, d_{ij} ($i, j = 1, 2, \dots, n$) 表示城市 i 与 j 之间的距离, 它是已知的 (或者城市的坐标集合为已知, d_{ij} 即为城市 i 与 j 之间的欧几里德距离)。TSP 的目的是找到从某个城市 c_i 出发, 访问所有城市且只访问一次, 最后回到 c_i 的最短封闭路线。

蚂蚁系统求解TSP

1. 路径构建

每只蚂蚁都随机选择一个城市作为其出发城市，并维护一个路径记忆向量，用来存放该蚂蚁依次经过的城市。蚂蚁在构建路径的每一步中，按照一个随机比例规则选择下一个要到达的城市。

- **定义** AS 中的随机比例规则 (random proportional) : 对于每只蚂蚁 k ，路径记忆向量 R^k 按照访问顺序记录了所有已经走过的城市序号。设蚂蚁 k 当前所在城市为 i ，则其选择城市 j 作为下一个访问对象的概率为：

蚂蚁系统求解TSP

信息素数量

启发式距离

$$P_k(i, j) = \begin{cases} \frac{[\tau(i, j)]^\alpha \cdot [\eta(i, j)]^\beta}{\sum_{u \in J_k(i)} [\tau(i, u)]^\alpha \cdot [\eta(i, u)]^\beta}, & \text{if } j \in J_k(i) \\ 0, & \text{otherwise} \end{cases}$$

其中， $J_k(i)$ 表示从城市 i 可以直接到达的且又不在蚂蚁访问过的城市序列 R^k 中的城市集合。 $\eta(i, j)$ 是一个启发式信息，通常由 $\eta(i, j) = 1/d_{ij}$ 直接计算。 $\tau(i, j)$ 表示边 (i, j) 上的信息素量。由公式可知，长度越短、信息素浓度越大的路径被蚂蚁选择的概率越大。 α 和 β 是两个预先设置的参数，用来控制启发式信息与信息素浓度作用的权重关系。

2. 信息素更新

- 在算法初始化时，问题空间中所有的边上的信息素都被初始化为 τ_0 。如果 τ_0 太小，算法容易早熟，即蚂蚁很快就全部集中在一条局部最优的路径上。反之，如果 τ_0 太大，信息素对搜索方向的指导作用太低，也会影响算法性能。对AS来说，一般情况下使用 $\tau_0 = m/C^m$ ， m 是蚂蚁的个数， C^m 是由贪婪算法构造的路径的长度。
- 所有蚂蚁构建完路径后，算法对所有的路径进行全局信息素的更新。在AS的ant-cycle版本中，更新是在全部蚂蚁均完成了路径的构造后才进行的，信息素的浓度变化与蚂蚁在这一轮中构建的路径长度相关。

也可以在每个蚂蚁走过之后更新

信息素的更新有两个步骤:

在每一个蚂蚁都走过，更新完信息素之后，进行一次的更新

- 首先，每一轮过后，问题空间中的所有路径上的信息素都会发生蒸发，我们为所有边上的信息素乘上一个小于1的常数，表示蒸发以后的余量。信息素蒸发是自然界本身固有的特征，在算法中能够避免信息素的无限积累，使得算法可以快速丢弃之前构建过的较差的路径。
- 随后，所有蚂蚁根据本轮中自己构建的路径长度，在它们经过的边上释放信息素。蚂蚁构建的路径越短、释放的信息素就越多，一条边被蚂蚁爬过的次数越多、它所获得的信息素也越多。AS 中城市 i 与城市 j 的相连边上的信息素量 $\tau(i, j)$ 的更新公式为:

$$\tau(i, j) \leftarrow (1 - \rho) \cdot \tau(i, j) + \sum_{k=1}^m \Delta \tau_k(i, j)$$

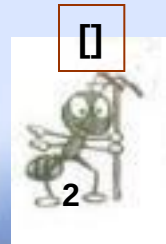
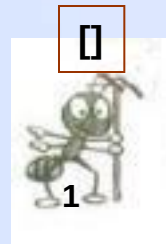
信息素的蒸发率

$$\Delta \tau_k(i, j) = \begin{cases} 1/C_k, & \text{if } (i, j) \in R^k \\ 0, & \text{otherwise} \end{cases}$$

m 个蚂蚁在边
(i, j)上释放的
信息素总和

- m 是蚂蚁个数; ρ 是信息素的蒸发率, 规定 $0 < \rho \leq 1$, 在 AS 中通常设置为 $\rho = 0.5$
- $\Delta \tau_k(i, j)$ 是第 k 只蚂蚁在它经过的边上释放的信息素量, 它等于蚂蚁 k 本轮构建路径长度的倒数
- C_k 表示路径长度, 它是 R^k 中所有边的长度之和。

A simple TSP example



路径向量



A

B

C

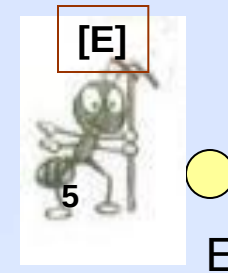
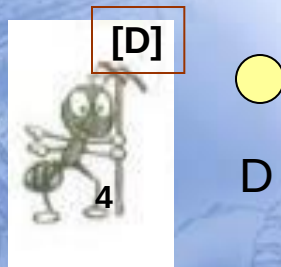
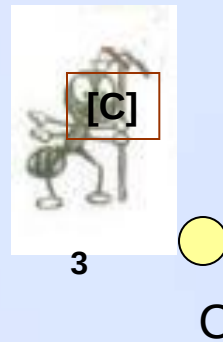
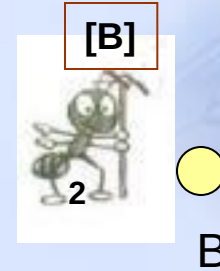
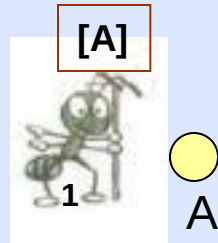
D

E



$$d_{AB}=100; d_{BC}=60; \dots; d_{DE}=150$$

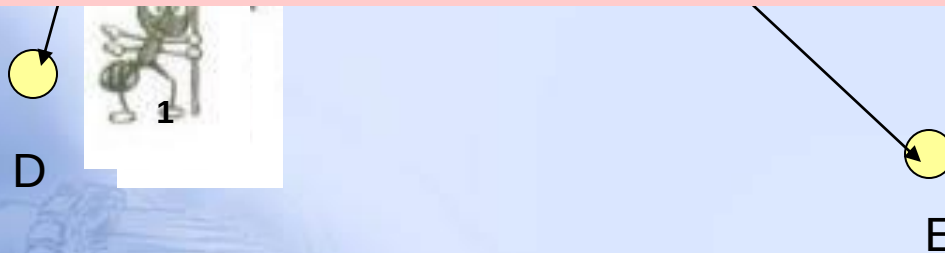
Iteration 1



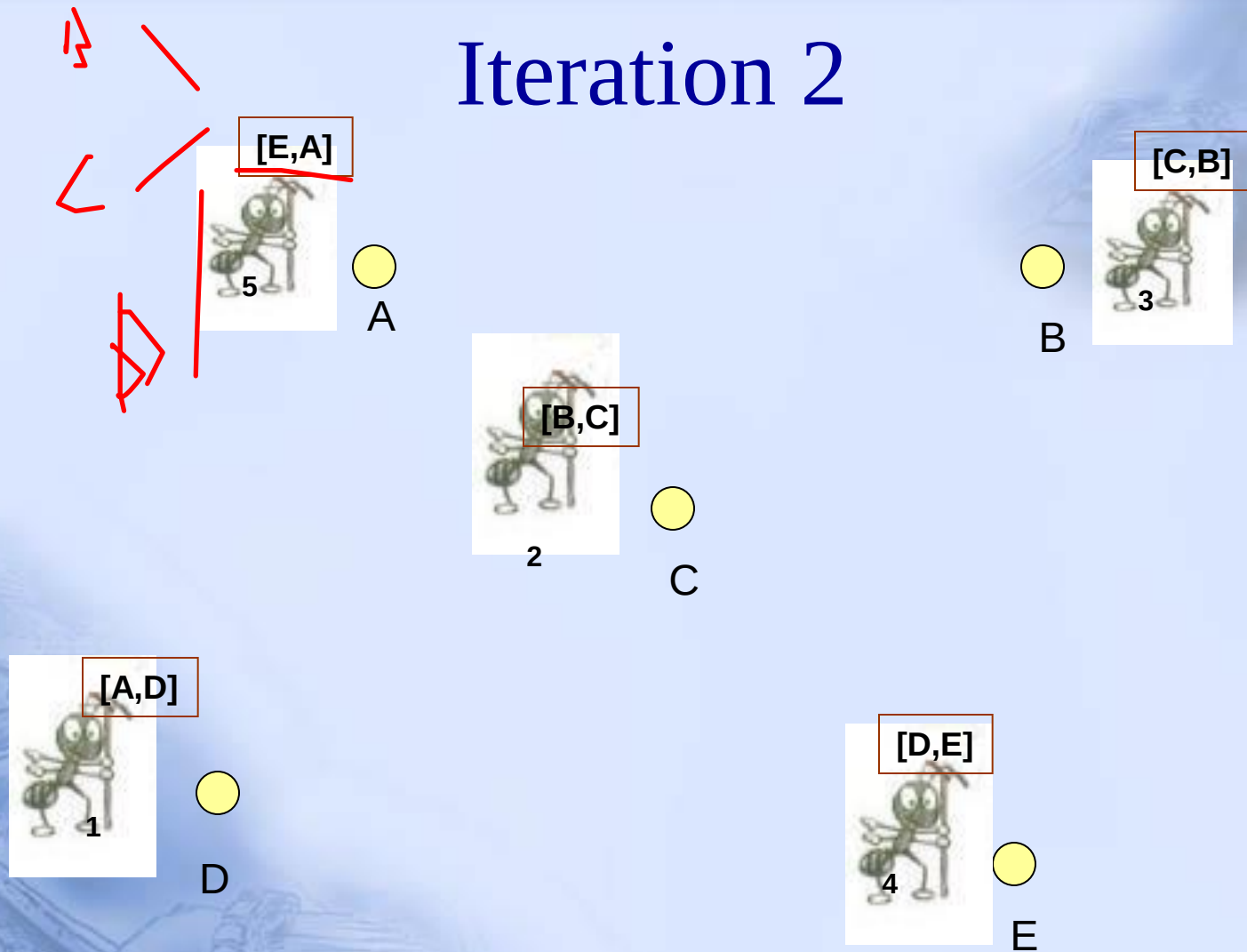
如何选择下一个城市?



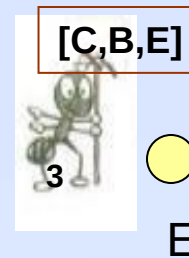
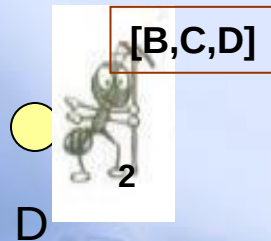
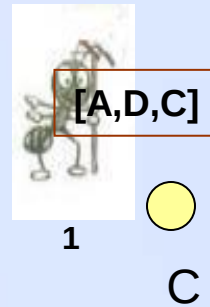
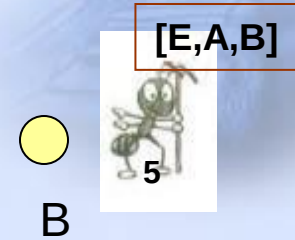
$$P_k(i, j) = \begin{cases} \frac{[\tau(i, j)]^\alpha \cdot [\eta(i, j)]^\beta}{\sum_{u \in J_k(i)} [\tau(i, u)]^\alpha \cdot [\eta(i, u)]^\beta}, & \text{if } j \in J_k(i) \\ 0, & \text{otherwise} \end{cases}$$



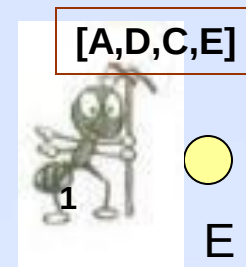
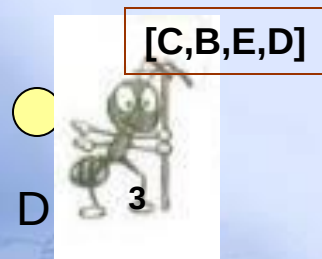
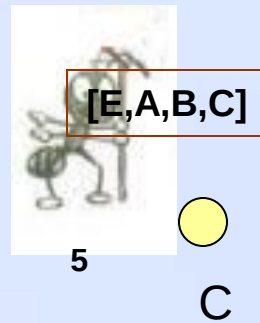
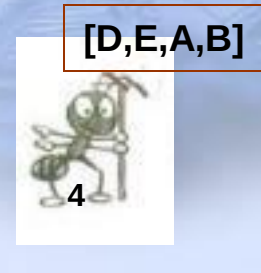
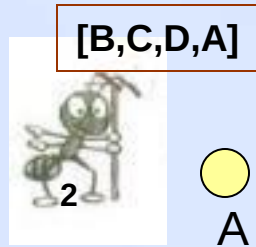
Iteration 2



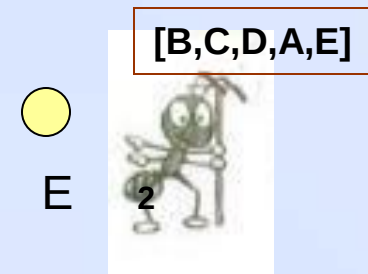
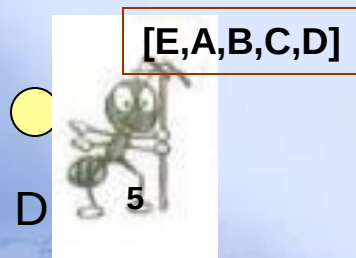
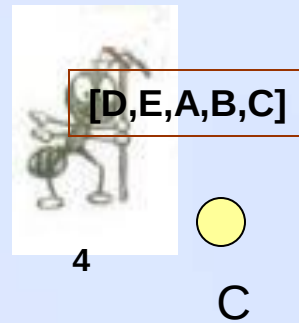
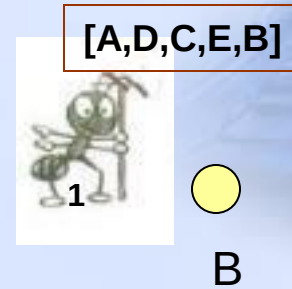
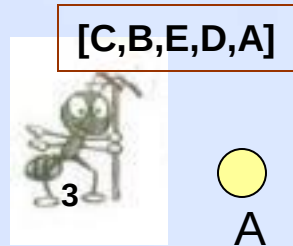
3



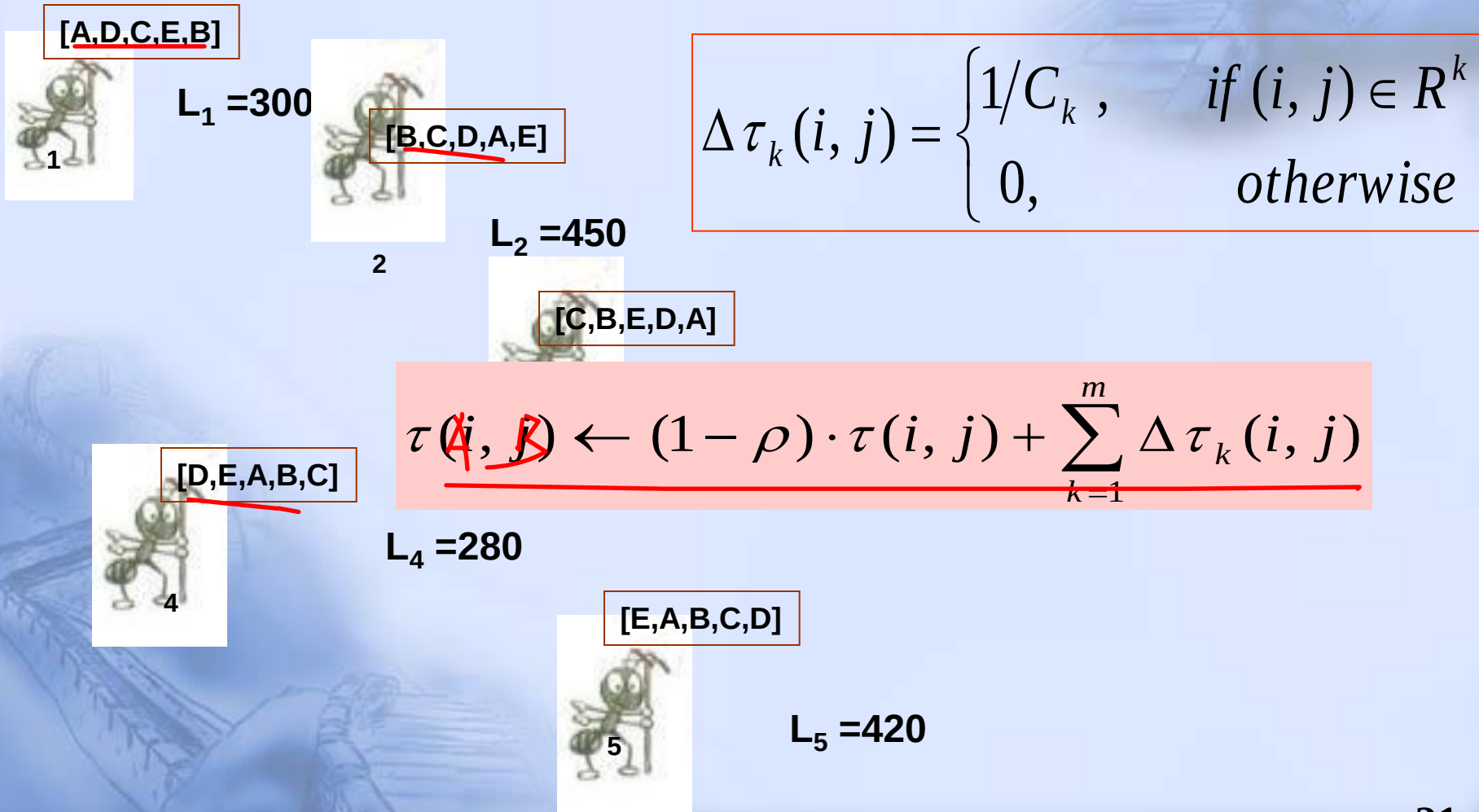
Iteration 4

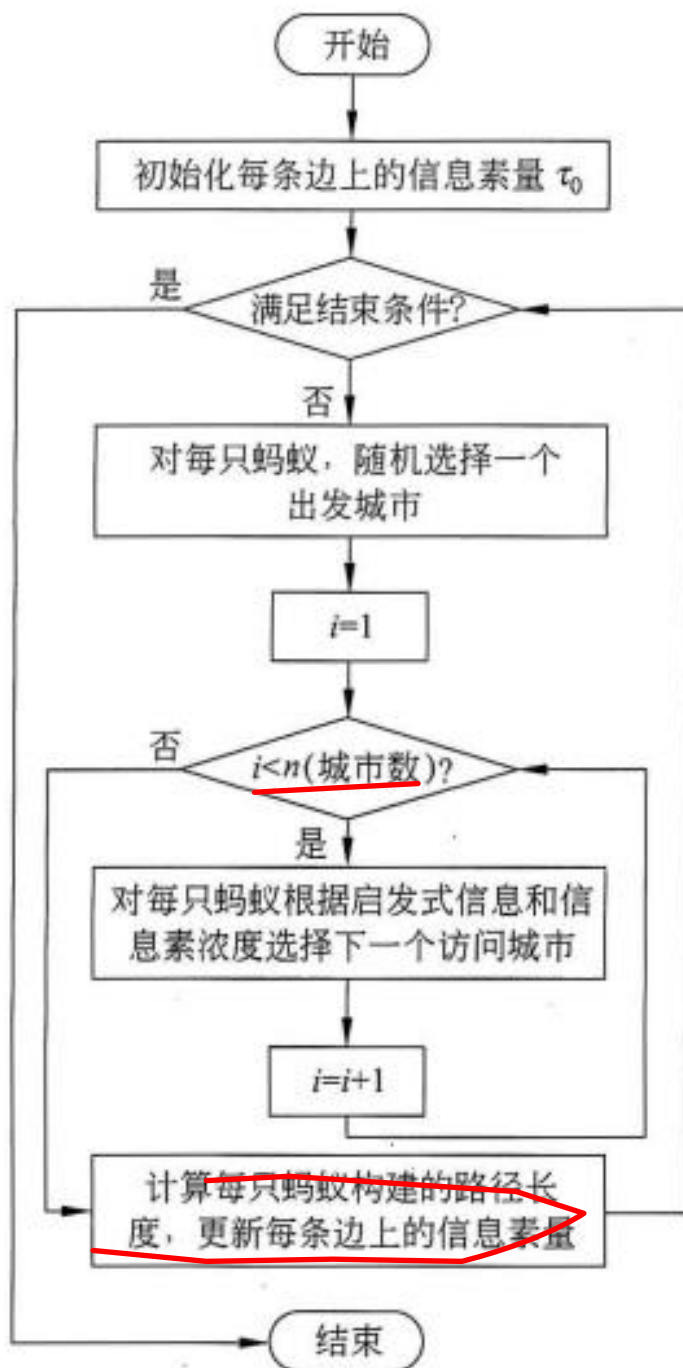


Iteration 5



信息素释放





//功能: 蚂蚁系统伪代码

//说明: 本例以求解 TSP 问题为目标

//参数: N 为城市规模

procedure AS

for each edge

set initial pheromone value τ_0 .

end for

while not stop

for each ant k

randomly choose an initial city.

for $i=1$ to n

choose next city j with the probability given by Eq. (5.1).

end for

end for

compute the length C_k of the tour constructed by the k th ant.

for each edge

update the pheromone value by Eq. (5.2).

end for

end while

print result.

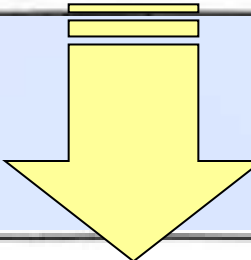
end procedure

例 5.1 给出用蚁群算法求解一个四城市的 TSP 问题的执行步骤, 四个城市 A 、 B 、 C 、 D 之间的距离矩阵如下:

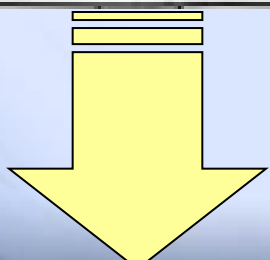
$$W = d_{ij} = \begin{bmatrix} \infty & 3 & 1 & 2 \\ 3 & \infty & 5 & 4 \\ 1 & 5 & \infty & 2 \\ 2 & 4 & 2 & \infty \end{bmatrix}$$

假设蚂蚁种群的规模 $m=3$, 参数 $\alpha=1, \beta=2, \rho=0.5$

步骤：初始化。首先使用贪婪算法得到路径 $(ACDBA)$ ，则 $C^m = f(ACDBA) = 1+2+4+3=10$ 。求得 $\tau_0 = m/C^m = 3/10 = 0.3$ 。初始化所有边上的信息素 $\tau_{ij} = \tau_0$ 。



步骤 2.1：为每只蚂蚁随机选择出发城市，假设蚂蚁 1 选择城市 A ，蚂蚁 2 选择城市 B ，蚂蚁 3 选择城市 D 。



步骤 2.2：为每只蚂蚁选择下一访问城市。
我们仅以蚂蚁 1为例，当前城市 $i=A$ ，可访问城市集合 $J_1(i)=\{B,C,D\}$ 。计算蚂蚁 1 选择 B,C,D 作为下一访问城市的概率：

$$A \Rightarrow \begin{cases} B: \tau_{AB}^{\alpha} \times \eta_{AB}^{\beta} = 0.3^1 \times (1/3)^2 = 0.033 \\ C: \tau_{AC}^{\alpha} \times \eta_{AC}^{\beta} = 0.3^1 \times (1/1)^2 = 0.3 \\ D: \tau_{AD}^{\alpha} \times \eta_{AD}^{\beta} = 0.3^1 \times (1/2)^2 = 0.075 \end{cases}$$

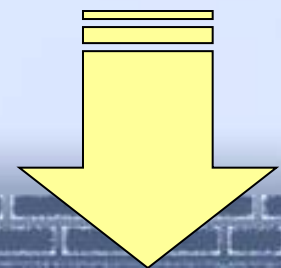
$$p(B) = 0.033 / (0.033 + 0.3 + 0.075) = 0.081$$

$$p(C) = 0.3 / (0.033 + 0.3 + 0.075) = 0.74$$

$$p(D) = 0.075 / (0.033 + 0.3 + 0.075) = 0.18$$

用轮盘赌法则选择下一访问城市。假设产生的随机数 $q = \text{random}(0,1) = 0.05$ ，则蚂蚁 1 将会选择城市 B 。

用同样的方法为蚂蚁 2 和 3 选择下一访问城市，假设蚂蚁 2 选择城市 D ，蚂蚁 3 选择城市 A 。



步骤 2.3 : 当前蚂蚁 1 所在城市 $i=B$, 路径记忆向量 $R^1=(AB)$, 可访问城市集合 $J_1(i)=\{C, D\}$ 。计算蚂蚁 1 选择 C, D 作为下一城市的概率:

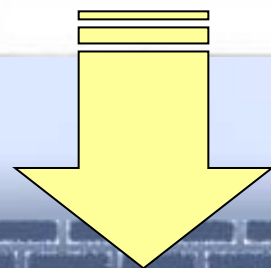
$$B \Rightarrow \begin{cases} C: \tau_{BC}^{\alpha} \times \eta_{BC}^{\beta} = 0.3^1 \times (1/5)^2 = 0.012 \\ D: \tau_{BD}^{\alpha} \times \eta_{BD}^{\beta} = 0.3^1 \times (1/4)^2 = 0.019 \end{cases}$$

$$p(C) = 0.012 / (0.012 + 0.019) = 0.39$$

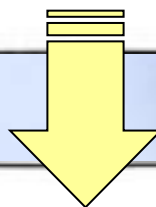
$$p(\underline{D}) = 0.019 / (0.012 + 0.019) = 0.61$$

用轮盘赌法则选择下一访问城市。假设产生的随机数 $q=\text{random}(0,1)=\underline{0.67}$, 则蚂蚁 1 将会选择城市 D 。

用同样的方法为蚂蚁 2 和 3 选择下一访问城市, 假设蚂蚁 2 选择城市 C , 蚂蚁 3 选择城市 D 。



步骤 2.4：实际上此时路径已经构造完毕，蚂蚁 1 构建的路径为 (ABDCA)。蚂蚁 2 构建的路径为 (BDCAB)。蚂蚁 3 构建的路径为 (DACBD)。



步骤 3：信息素更新。

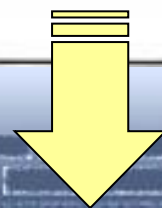
计算每只蚂蚁构建的路径长度： $C_1=3+4+2+1=10$, $C_2=4+2+1+3=10$, $C_3=2+1+5+4=12$ 。更新每条边上的信息素：

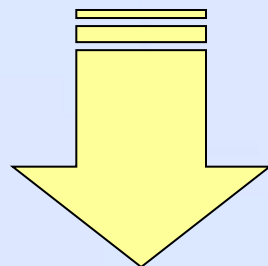
$$\tau_{AB} = (1-\rho) \times \tau_{AB} + \sum_{k=1}^3 \Delta \tau_{AB}^k = 0.5 \times 0.3 + (1/10 + 1/10) = 0.35$$

$$\tau_{AC} = (1-\rho) \times \tau_{AC} + \sum_{k=1}^3 \Delta \tau_{AC}^k = 0.5 \times 0.3 + (1/12) = 0.16$$

.....

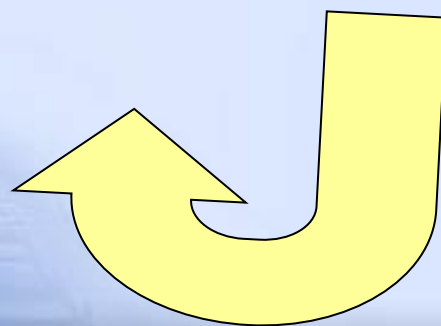
如上，根据公式 (5.2) 依次计算出问题空间内所有边更新后的信息素量。





步骤 4 :

如果满足结束条件，则输出全局最优结果并结束程序，
否则，转向步骤 2.1 继续执行。



蚁群算法的改进版本

精华蚂蚁系统(Elitist Ant System, EAS)

- AS 算法中, 信息素量的释放, 与其构建路径长度成反比。蚂蚁构建的路径越好, 属于路径的各个边上所获得的信息素就越多, 在以后的迭代中被蚂蚁选择的概率也就越大。当城市的规模较大时, 问题的复杂度呈指数级增长, 仅仅靠单一的信息素更新机制引导搜索偏向, 搜索效率有瓶颈。
- EAS是对基础 AS 的第一次改进, 它在原 AS 信息素更新的基础上, 增加了一个对至今最优路径的强化手段。在每轮信息素更新完毕后, 搜索到至今最优路径(T_b)的那只蚂蚁会为这条路径添加额外的信息素。

$$\tau(i, j) = (1 - \rho) \cdot \tau(i, j) + \sum_{k=1}^m \Delta \tau_k(i, j) + e \Delta_b(i, j)$$

$$\Delta \tau_k(i, j) = \begin{cases} (C_k)^{-1}, & (i, j) \in R^k \\ 0, & \text{其他} \end{cases}$$

$$\Delta_b(i, j) = \begin{cases} (C_b)^{-1}, & (i, j) \text{ 在路径 } T_b \text{ 上} \\ 0, & \text{其他} \end{cases}$$

在 EAS 中, 新增了 $\Delta_b(i, j)$, 并定义参数 e 作为 $\Delta_b(i, j)$ 的权值。
 C_b 是算法开始至今最优路径的长度。可见, EAS 在每轮迭代中
 为属于 T_b 的边 增加了额外的 e/C_b 的信息素量。

引入这种额外的信息素强化手段有助于更好地引导蚂蚁搜索的偏向,
 使算法更快收敛。

基于排列的蚂蚁系统(rank-based AS, AS_{rank})

- 有没有更好的一种信息素更新方式，它既能使 T_b 各边的信息素浓度得到加强，同时对其余边的信息素更新机制也有改善？
- AS_{rank} 就是这种改进版本，它在AS 的基础上给蚂蚁要释放的信息素量 $\Delta\tau_k(i, j)$ 加一个权值，进一步加大各边信息素量的差异。
- 在每一轮路径构建完以后，所有蚂蚁将按照所得路径的长短进行排名，只有生成了至今最优路径的蚂蚁和排名在前($\omega-1$)的蚂蚁才允许释放信息素，蚂蚁在边(i, j) 上释放的信息素量 $\Delta\tau_k(i, j)$ 的权值由蚂蚁的排名决定。

$$\tau(i, j) = (1 - \rho) \cdot \tau(i, j) + \sum_{k=1}^{\omega-1} (\omega - k) \Delta\tau_k(i, j) + \omega \Delta_b(i, j)$$

$$\Delta\tau_k(i, j) = \begin{cases} (C_k)^{-1}, & (i, j) \in R^k \\ 0, & \text{其他} \end{cases}$$

$$\Delta_b(i, j) = \begin{cases} (C_b)^{-1}, & (i, j) \text{ 在路径 } T_b \text{ 上} \\ 0, & \text{其他} \end{cases}$$

构建至今最优路径 T_b 的蚂蚁产生信息素的权值大小为 ω ，它将在 T_b 的各边上增加 ω/C_b 的信息素量，也就是说，路径 T_b 将获得最多的信息素量。

其余的，在本次迭代中排名第 $k(k=1, 2, \dots, \omega-1)$ 的蚂蚁将释放 $(\omega-k)/C_k$ 的信息素。排名越前的蚂蚁释放的信息素量越大，权值 $(\omega-k)$ 对不同路径的信息素浓度差异起到了一个放大的作用

最大最小蚂蚁系统(MAX-MIN Ant System, MMAS)

先思考两个问题:

- 问题一: 对于大规模的 TSP , 由于搜索蚂蚁的个数有限, 而初始化时蚂蚁的分布是随机的, 这会不会造成蚂蚁只搜索了所有路径中的小部分就以为找到了最好的路径, 所有的蚂蚁都很快聚集在同一路径上, 而真正优秀的路径并没有被探索到呢?
- 问题二: 当所有蚂蚁都重复构建着同一条路径的时候, 意味着算法已经进入停滞状态。有没有办法利用算法停滞后的迭代过程进一步搜索以保证找到更接近真实目标的解呢?

最大最小蚂蚁系统(MAX-MIN Ant System, MMAS)

在 AS 算法的基础上进行了四项改进。

- 只允许**迭代最优蚂蚁** (本次迭代构建出最短路径的蚂蚁), 或者**至今最优蚂蚁**释放信息素。
- 信息素量大小的**取值范围**被限制在一个区间内。
- 信息素**初始值**为信息素取值区间的上限, 并伴随一个较小的信息素蒸发速率。
- 每当系统进入停滞状态, 问题空间内所有边上的信息素量都会被**重新初始化**。

改进(1)

- 借鉴了精华蚂蚁系统，但又有细微的不同。在 EAS 中，只允许至今最优的蚂蚁释放信息素。
- 只使用至今最优更新规则进行信息素的更新，**搜索的导向性**很强，算法会很快收敛到至今最优路径 T_b 附近;反之，如果只使用迭代最优更新规则，则算法的探索能力会得到增强，但**收敛速度**会下降
- 实际应用中，迭代最优更新规则和至今最优更新规则在 MMAS 中会被**交替使用**。
- 一种好的方式是，在算法迭代过程中，逐渐加大至今最优更新的概率。

改进(2)

- 为了避免某些边上的信息素浓度增长过快，算法出现早熟现象，即所有蚂蚁都搜索一条较优而不是最优的路径，把信息素量的大小限定在一个取值范围 $[\tau_{min}, \tau_{max}]$ 内
- 蚂蚁是依据启发式信息和信息素浓度选择下城市节点的，其中启发式信息取值范围已经确定，当信息素浓度也被限制在一个范围内以后，选择下一城市的概率 $p_k(i, j)$ 也将被限制在一个区间内。
- 假设区间为 $[p_{min}, p_{max}]$ ，只要满足 $0 < p_{min} < p_k(i, j) < p_{max}$ ，就可以确定算法已经有效避免了陷入停滞状态的可能性。

改进(3)

- 初始化时所有边上的信息素均被初始化为 τ_{max} 的估计值，且信息素蒸发速率非常小(一般将 ρ 置为 0.02)。
- 不同边上的信息素浓度差异只会缓慢地增加，因此在算法的初始阶段，MMAS 具有更强的探索能力。
- 增强算法在初始阶段具有的较强探索能力有助于蚂蚁进行全局范围内的搜索，随后再逐渐缩小搜索范围，最后定格在一条全局最优路径上。

改进(4)

- 当算法接近或是进入停滞状态时，所有边上的信息素浓度都将被重新初始化，利用系统进入停滞状态后的迭代周期继续进行搜索，使算法具有更强的全局寻优能力。

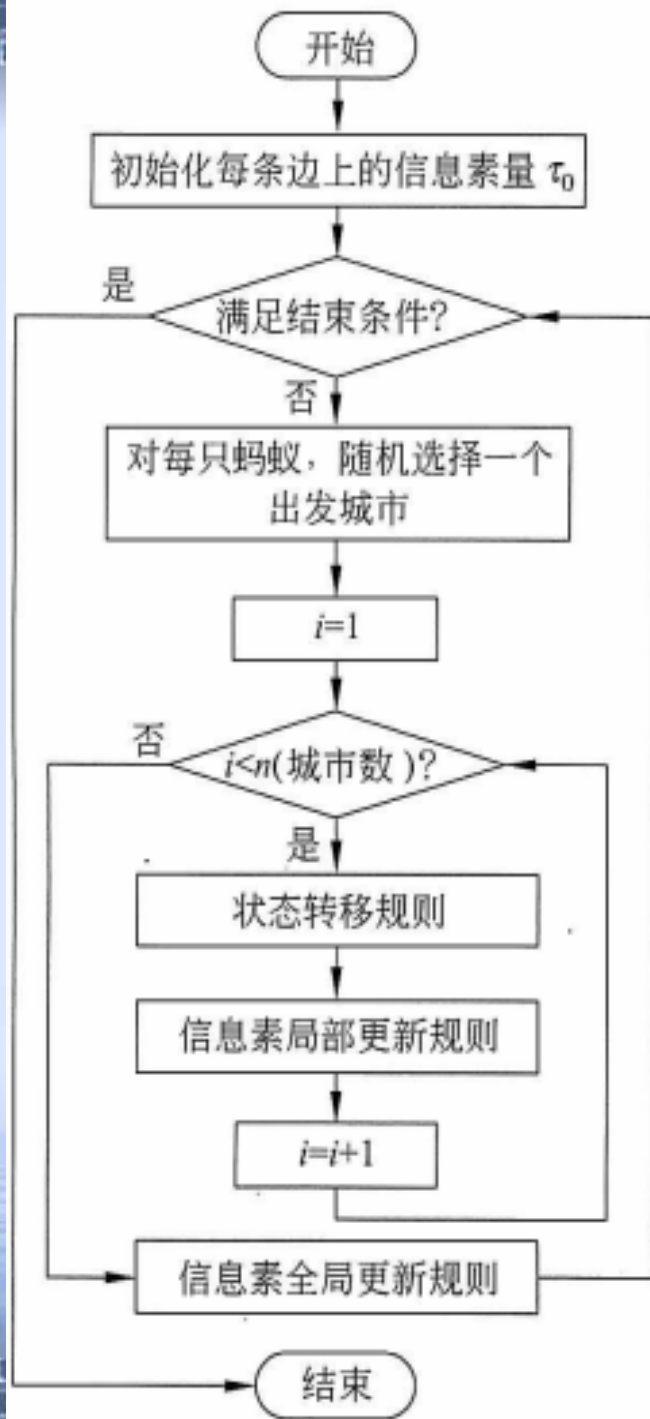
蚁群系统(Ant Colony System, ACS)

ACS 与蚂蚁系统的不同主要体现在三个方面:

- 使用一种伪随机比例规则(pseudorandom proportional)选择下一城市节点, 建立开发当前路径与探索新路径之间的平衡。
- 信息素全局更新规则只在属于至今最优路径的边上蒸发和释放信息素。
- 新增信息素局部更新规则, 蚂蚁每次经过空间内的某条边, 它都会去除该边上一定量的信息素, 以增加后续蚂蚁探索其余路径的可能性。

蚁群系统(Ant Colony System, ACS)

- 将 m 只蚂蚁随机或是均匀地分布在 n 个城市上，然后每只蚂蚁根据**状态转移规则**确定下一步要去的城市。
- 每只蚂蚁都配有一张搜索禁忌表，单个蚂蚁在遍历过程中会在它们经过的路径上进行**信息素局部更新**。
- 在每轮所有的蚂蚁均完成汉密尔顿回路的构造后，需记录下这些回路中最短的一条，并按照**信息素全局更新规则**增加这条路径上的信息素。
- ACS 中有**状态转移规则**、**信息素全局更新规则**、**信息素局部更新规则**三大核心规则。



(1)状态转移规则

- 定义 5.2 ACS 中的伪随机比例规则 (pseudorandom proportional): 对于每只蚂蚁走, 路径记忆向量 R^k 按照访问顺序记录了所有是已经经过的城市序号。设蚂蚁走当前所在城市为 i , 则下一个访问城市

$$j = \begin{cases} \arg \max_{j \in J_k(i)} \{ \underline{\tau(i, j)} \underline{[\eta(i, j)]^\beta} \}, & q \leq q_0 \\ S, & \text{其他} \end{cases}$$

$J_k(i)$ 表示城市集合, q_0 是一个 $[0, 1]$ 区间内的参数, 当随机数

- $q \leq q_0$ 时, 蚂蚁直接选择使启发式信息与信息素量的指数乘积最大的下一城市节点, 即开发 (exploitation);
- $q > q_0$ 时, ACS 将和各种 AS 算法一样使用轮盘赌选择策略, 即偏向探索 (biased exploration)。

位于城市 i 的蚂蚁 k 选择城市 j 作为下一个访问对象的概率:

$$p_k(i, j) = \begin{cases} \frac{[\tau(i, j)][\eta(i, j)]^\beta}{\sum_{u \in J_k(i)} [\tau(i, u)][\eta(i, u)]^\beta}, & j \in J_k(i) \\ 0, & \text{其他} \end{cases}$$

- 在 ACS 的状态转移规则中, 蚂蚁选择当前最优移动方向的概率为 q_0 , 同时以 $(1 - q_0)$ 的概率有偏向地搜索各条边。
- 通过调整 q_0 , ACS 能有效调节"开发"与"探索"之间的平衡, 以决定算法是集中开发最优路径附近的区域, 还是探索其他的区域。

(2)信息素全局更新规则

- 只有至今最优蚂蚁(构建出了从算法开始到当前迭代中最短路径的蚂蚁)被允许释放信息素,
- 在每轮的迭代中, 所有蚂蚁均构建完路径后, 信息素全局更新规则才被使用:

$$\tau(i, j) = (1 - \rho) \cdot \tau(i, j) + \rho \cdot \Delta\tau_b(i, j), \quad \forall (i, j) \in T_b$$

- $\Delta\tau_b(i, j) = 1/C_b$, 不论是信息素的蒸发还是释放, 都只在至今最优路径的边上进行。
- 参数 ρ 代表信息素蒸发的速率, 新增加的信息素 $\Delta\tau_b(i, j)$ 乘上系数 ρ 以后, 更新后的信息素浓度被控制在旧信息素量与新释放的信息素量之间, 用一种更简单的方式实现了 **MMAS** 算法中对信息素量取值范围的限制。

(3)信息素局部更新规则

ACS 的一项重大改进就是引入了信息素局部更新规则。在路径构建过程中，对每一只蚂蚁，每当其经过一条边 (i, j) 时，它立刻对这条边的信息素进行更新：

$$\tau(i, j) = (1 - \xi) \cdot \tau(i, j) + \xi \cdot \tau_0$$

- 其中， ξ 是信息素局部挥发速率，满足 $0 < \xi < 1$ 。 τ_0 是信息素的初始值。
- 通过实验发现， ξ 为 0.1， τ_0 取值为 $1/(nC^m)$ 时，算法对大多数实例有着非常好的性能。其中 n 为城市个数， C^m 是由贪婪算法构造的路径的长度。

由于 $\tau_0 = \underline{1/(nC^m)} \leq \tau(i, j)$

- 公式(5.8) 所计算出来的更新后的信息素相比更新前减少了，也就是说，信息素局部更新规则作用于某条边上会使得这条边被其他蚂蚁选中的概率减少。
- 这种机制大大增加了算法的探索能力，后续蚂蚁倾向于探索未被使用过的边，有效地避免了算法进入停滞状态。

蚁群算法的其它改进版本

近似非确定性树搜索

(Approximate Nondeterministic Tree Search, ANTS)

在三个方面对**AS** 算法进行了修改:

- 它使用部分解的完全代价估计的下界来计算各边的**启发式信息**;
- 它使用**加法**而非乘法来实现**启发式距离与信息素的结合**;
- 没有直观的信息素蒸发步骤, 信息素增加量的**计算公式**也与之前各种 **ACO** 算法很大不同。

多态蚁群算法

(Polymorphic Ant Colony Algorithm, PACA)

将蚁群中的蚂蚁分为三类: 侦查蚁、搜索蚁、工蚁

- 侦查蚁的任务是在算法初期以每个城市为中心做局部区域观察, 并将侦查结果与已有的先验知识结合, 生成侦查素, 问题空间内各边的初始信息素量与侦查素有关。
- 搜索蚁是各类传统蚁群算法通常所指的蚂蚁, 它根据启发式信息和信息素选择下一城市节点, 直至构建出最佳路径。
- 工蚁与算法最优路径的寻找无关。

带聚类处理的蚁群算法

(Clustering Processing Ant Colony Algorithm, CPACA)

- 将大规模的 **TSP** 分解为**小规模子问题**分别用蚁群算法进行求解，再将**各个小规模子问题的解合并起来**，便可高效地得到原待求解问题的解。
- **CPACA** 算法先对城市空间进行聚类处理，在每一个类中用蚁群算法求得**类内最短路径**，随后将各个类的中心看成一个 **TSP** 也用蚁群算法求解得**类间最短路径**，最后确定每个类的**边界城市**，通过边界城市将各个类连接起来。

连续正交蚁群算法

(Continuous Orthogonal Ant Colony , COAC)

连续空间的蚁群算法

- 通过在问题空间内自适应地选择和调整一定数量的区域，并利用蚂蚁在这些区域内进行正交搜索、在区域间进行状态转移，并更新各个区域的信息素来搜索问题空间中的最优解。
- **COAC** 的基本思想是利用正交试验的方法将连续空间离散化。

蚂蚁数目 m

- ✓ 影响着算法的搜索能力和计算量。
- 蚂蚁数目过多时, 每轮迭代的计算量大、且被搜索过的路径上信息素变化比较平均, 此时算法的全局随机搜索能力得到增强, 但收敛速度减慢。
- 蚂蚁数目过少时, 算法的探索能力变差, 容易出现早熟现象。特别是当问题规模很大时, 算法的全局寻优能力会变得十分糟糕。
- Dorigo 等人的实验^[84]表明, 在用 AS、EAS、AS_{rank} 和 MMAS 求解 TSP 问题时, m 取值等于城市数目 n 算法有较好性能; 而对于 ACS, $m=10$ 比较合适

信息素权重 α 与启发式距离权重 β

- ✓ 决定算法搜索的导向,影响算法的搜索能力。
- α 越小,最邻近城市被选中的概率越大,蚂蚁越注重“眼前利益”。
 $\alpha=0$ 时,算法等同于随机贪婪算法。
- β 越小,蚂蚁越倾向于根据信息素浓度确定路径,算法收敛越快。
 $\beta=0$ 时,构建出的最优路径与实际目标有着较大差异,算法的性能比较糟糕。
- Dorigo 等人的实验^[84]表明,在各类 ACO 算法中设置
 $\alpha=1, \beta=2\sim5$ 比较合适

信息素挥发因子 ρ

- ✓ 影响蚂蚁个体之间相互影响的强弱，关系到算法的全局搜索能力和收敛速度。
- ρ 较大时，信息素挥发速率大，那些从未被蚂蚁选择过的边上的信息素急剧减小到接近 0，降低算法的全局探索能力。
- ρ 较小时，算法具有较高的全局搜索能力，但是由于各个路径的信息素浓度差距拉大较慢，算法收敛速度较慢。
- Dorigo 等人的实验^[84] 表明，
对于 AS 和 EAS, $\rho=0.5$; 对于 AS_{rank} , $\rho=0.1$;
对于 MMAS, $\rho=0.02$; 对于 ACS, $\rho=0.1$,
算法的综合性能较高

初始信息素量 τ_0

- ✓ 决定算法在初始化阶段的探索能力,影响算法的收敛速度。
- τ_0 太小,未被蚂蚁选择过的边上信息素太少,蚂蚁很快就全部集中在一条局部最优的路径上,算法容易早熟。
- τ_0 太大,信息素对搜索方向的引导能力增长得十分缓慢,算法收敛慢。
- Dorigo 等人的实验^[84]推荐:
对于 AS, $\tau_0 = m/C^m$; 对于 EAS, $\tau_0 = (e + m)/\rho C^m$;
对于 AS_{rank}, $\tau_0 = 0.5r(r-1)/\rho C^m$; 对于 MMAS, $\tau_0 = 1/\rho C^m$;
对于 ACS, $\tau_0 = 1/nC^m$

终止条件

- ✓ 决定算法运行何时结束,由具体的应用和问题本身确定。
- 将最大循环数设定为 500、1000、5000,或者最大的函数评估次数,等等。
- 也可以使用算法求解得到一个可接受的解作为终止条件。
- 或者是当算法在很长一段迭代中没有得到任何改善时,可以终止算法

蚁群优化算法中其他一些经典参数的经验设置

参 数	参考设置 ✓ 表示参数的意义 • 表示参数的经验设置
释放信息素的蚂蚁个数 ω	✓ 影响算法的全局搜索能力和收敛速度。 • 在 AS_{rank} 中, 参数 ω 设置为 $\omega=6$
进化停滞判定代数 rs	✓ 影响算法的全局搜索能力。 • 在 MMAS 中, 参数 rs 设置为 $rs=25$
信息素局部挥发因子 ξ	✓ 决定蚂蚁搜索时相互影响的强度, 影响算法的全局搜索能力。 • 在 ACS 中, 参数 ξ 设置为 $\xi=0.1$
伪随机因子 q_0	✓ 决定算法“开发”与“探索”的相互关系, 影响算法的全局搜索能力和收敛速度。 • 在 ACS 中, 参数 q_0 设置为 $q_0=0.9$